VERIFIED EXECUTION / SUBMISSION

Blog Application - Cloud Execution Report

React frontend, Express API and persistent MySQL on Amazon EKS. The complete release workflow was executed on the personal AWS account in Mumbai. This report replaces the earlier local-only submission for cloud evaluation.

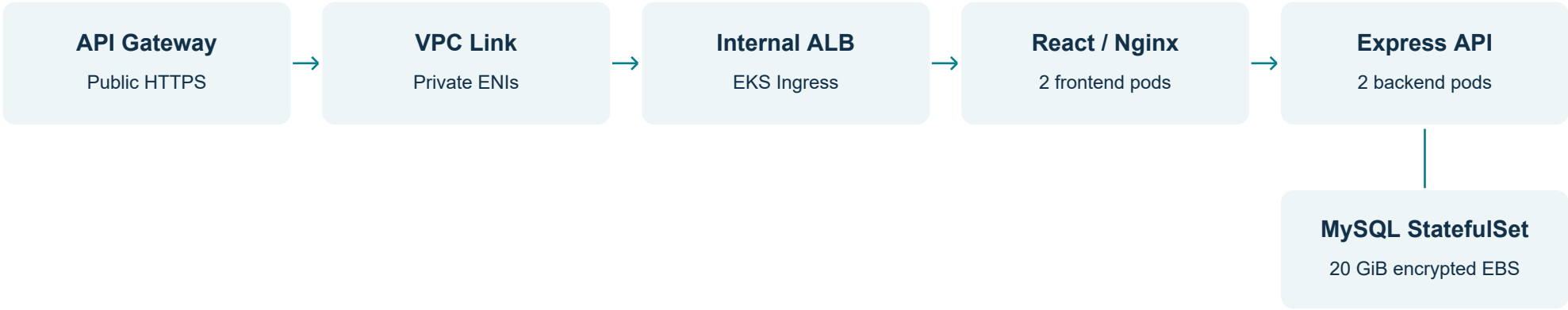
Submission item	Verified value
Application URL	https://1c3j649rgg.execute-api.ap-south-1.amazonaws.com
GitHub repository	https://github.com/praveenpreni/BlogReact
Document URL	https://github.com/praveenpreni/BlogReact/blob/main/docs/Cloud_Execution_Report.pdf
Acceptance release	Jenkins BlogApp/main #14 - SUCCESS; 89.04 seconds
Exact execution source	00bfc302acd73933912fd12579b72885081d22cb
Immutable image tag	00bfc302acd7-14
Private Jenkins build URL	http://127.0.0.1:18080/job/BlogApp/job/main/14/

Acceptance outcomes: 10 tests passed; quality gate passed; both container image gates passed; images published to both registries; public write/read/comment validation passed. The rolling acceptance release served 196 / 196 HTTP 200 responses.

Screenshots are genuine browser captures. Tables reproduce attached command/API results. UTC timestamps in raw records and Asia/Kolkata timestamps in browser views refer to the same execution. Administrative localhost links require the operator's authenticated forwarding session.

Three tiers with a private application origin

Internet clients use an AWS-managed HTTPS endpoint. API Gateway reaches the internal ALB through a VPC Link. Kubernetes Services connect frontend, backend and MySQL. The private API Gateway-to-ALB hop uses HTTP; public HTTPS terminates at API Gateway.



Layer	Executed controls
EKS / workers	Kubernetes 1.35; two private t3.large workers; two Availability Zones; IMDSv2; encrypted disks.
Platform services	VPC CNI NetworkPolicy enforcement, EBS CSI, metrics-server, AWS Load Balancer Controller.
Private CI host	t3.xlarge with encrypted 100 GiB disk; no inbound security-group rules; SSM administration.
CI tools	Jenkins 2.580.1; SonarQube Community Build 26.10.0.133078 + PostgreSQL 17; Nexus 3.97.2.
Observability	Prometheus/Grafana/Alertmanager on persistent storage; Kubernetes application logs; CloudWatch EKS API/audit/authenticator logs.

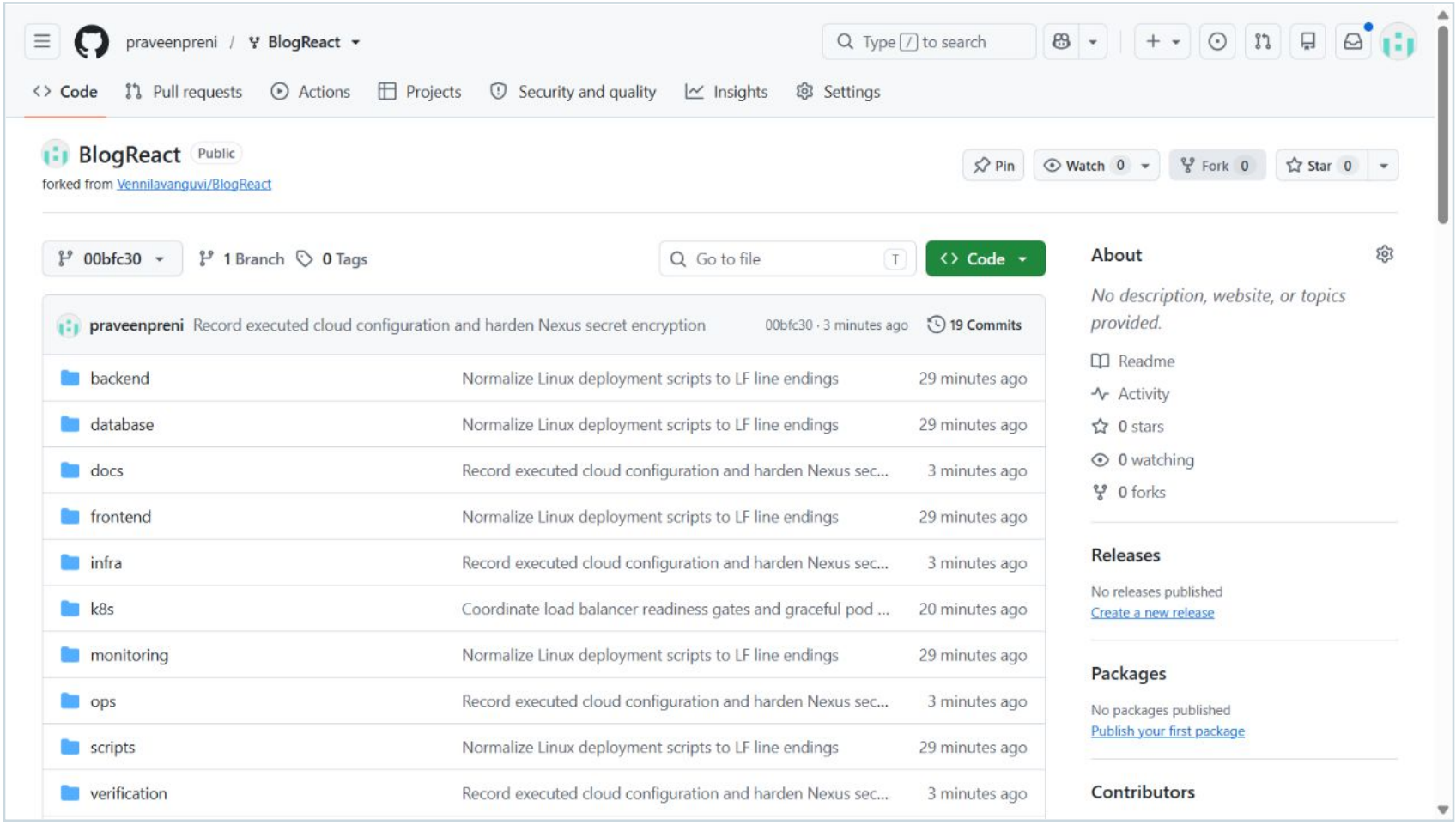
A custom domain was unnecessary. CloudFront distribution creation was blocked by AWS account verification; API Gateway provides the executed HTTPS path. The unused CloudFront origin was removed.

Requirement-to-evidence map

Brief	Requirement	Evidence	Recorded result
1	Working three-tier application	E08	Public register/login/post/read/comment operations passed.
2	Production-oriented Docker builds	E02	Multi-stage images, .dockerignore, non-root users, health checks; actual runtime inspection.
3	Persistent and secure database	E03, E09, E15	MySQL StatefulSet; encrypted PVC; schema initialization; recreation and restore verified.
4	Nexus and ECR	E05	Both versioned frontend/backend images present in both registries and used by EKS.
5	SonarQube and gate enforcement	E04	Current analysis passed; isolated missing-coverage fixture rejected before deployment.
6	Trivy image scanning	E06	HIGH/CRITICAL gate passed after remediation; vulnerable fixture rejected.
7	EKS deployment resources	E07	Deployments, StatefulSet, Services, Ingress, ConfigMap, Secret and PVC running.
8	Kubernetes production practices	E02, E07, E10	Replicas, probes, resources, service discovery, PDB/HPA and ALB readiness gates.
9	Configuration and secrets	E11	ConfigMap + Secret references; generated values retained privately in Secrets Manager.
10	Automated CI/CD	E03	Checkout, test, analysis, gates, build, publish, rollout and smoke validation succeeded.
11	Rolling deployment	E10	Commit-triggered release #12: 196 sampled requests, zero errors.
12	Prometheus / Grafana	E13	All six dashboard queries returned data; controlled alert fired and cleared.
13	Application logging	E14	Frontend, backend and MySQL kubectl log exports, plus EKS control-plane logging.
14	Security practices	E02, E06, E11, E12	Scoped IAM/RBAC, network denial tests, non-root execution and credential-free source.

The screenshot and execution commit match

Acceptance source: 00bfc302acd73933912fd12579b72885081d22cb
Baseline attribution: Vennilavanguvi/BlogReact, commit 420de1b1508d45addc2e76a3e9f24df274459bba. The repository screenshot is pinned to the acceptance source, not to a moving main branch.

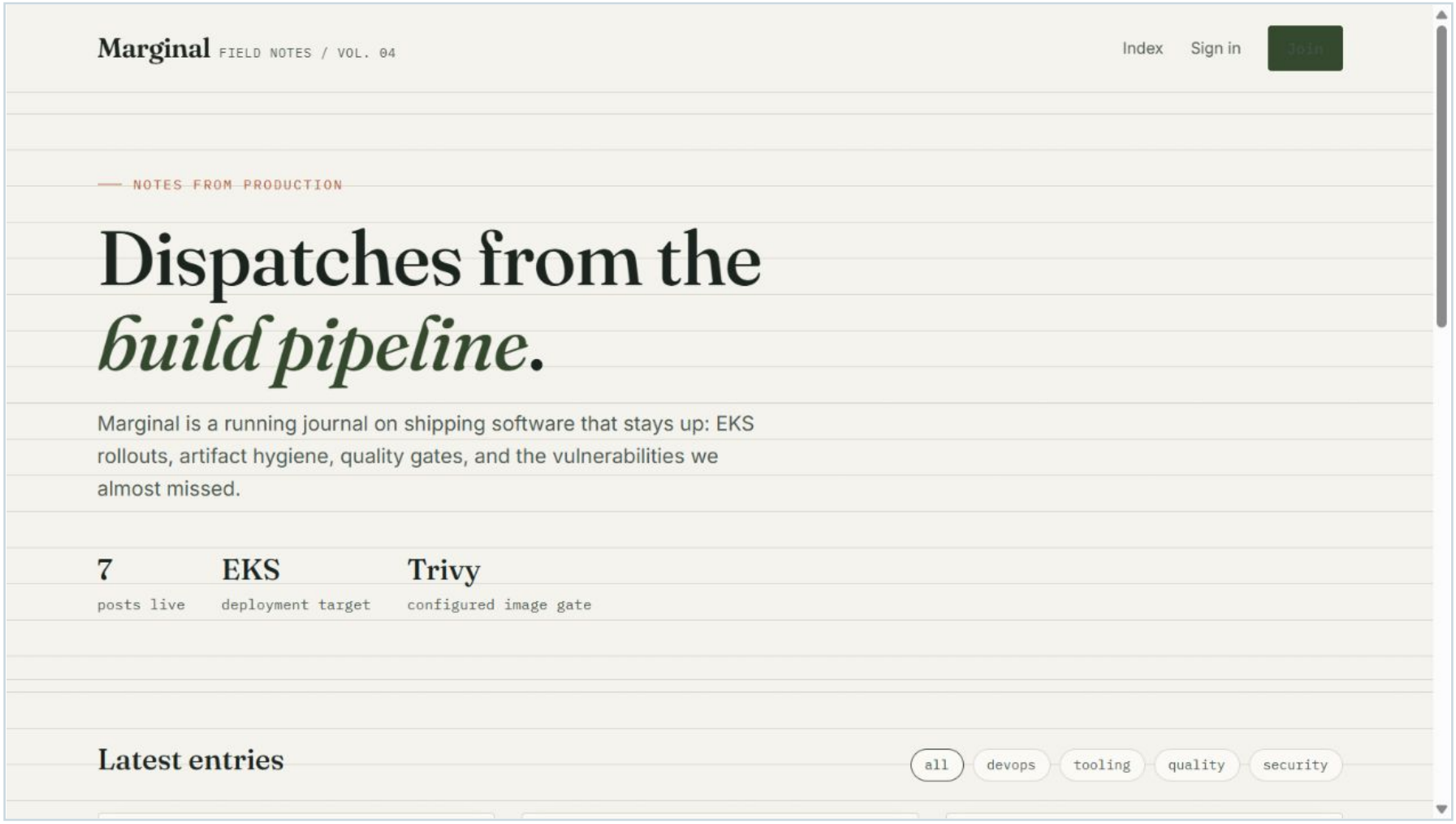


Pinned source view: https://github.com/praveenpreni/BlogReact/tree/00bfc302acd73933912fd12579b72885081d22cb

Report/evidence publication is a later documentation commit. It does not retroactively change the source commit, tag or image digests recorded for acceptance build #14.

Public HTTPS application

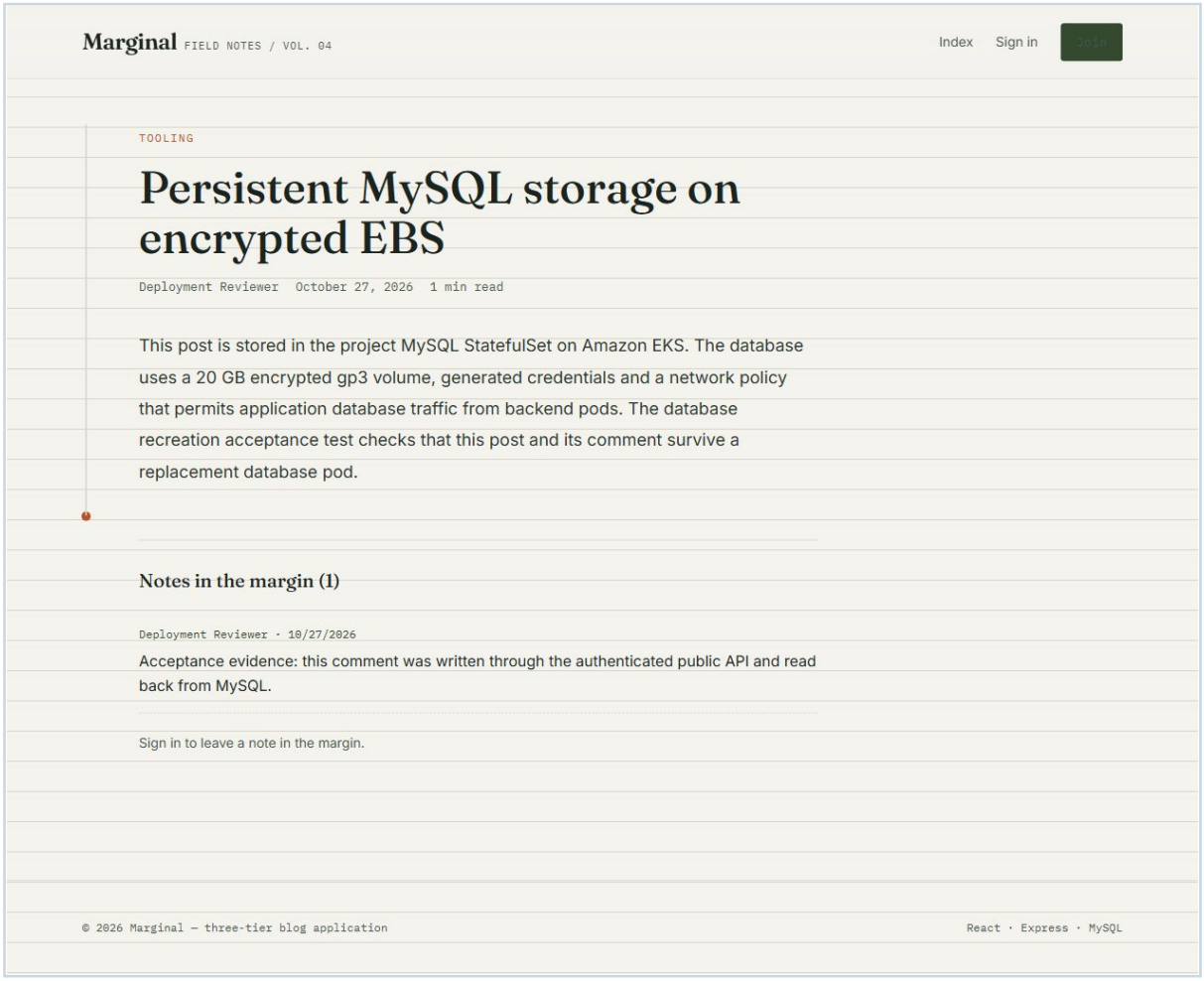
The deployed frontend loads data from the Express API and MySQL. Public API acceptance exercised registration, login, authenticated post creation, reading and comments. A generated review account was kept private.



Live application: https://1c3j649rgg.execute-api.ap-south-1.amazonaws.com. Initial API proof: raw/E08-application-uat.json.

Persisted post and comment read through the frontend

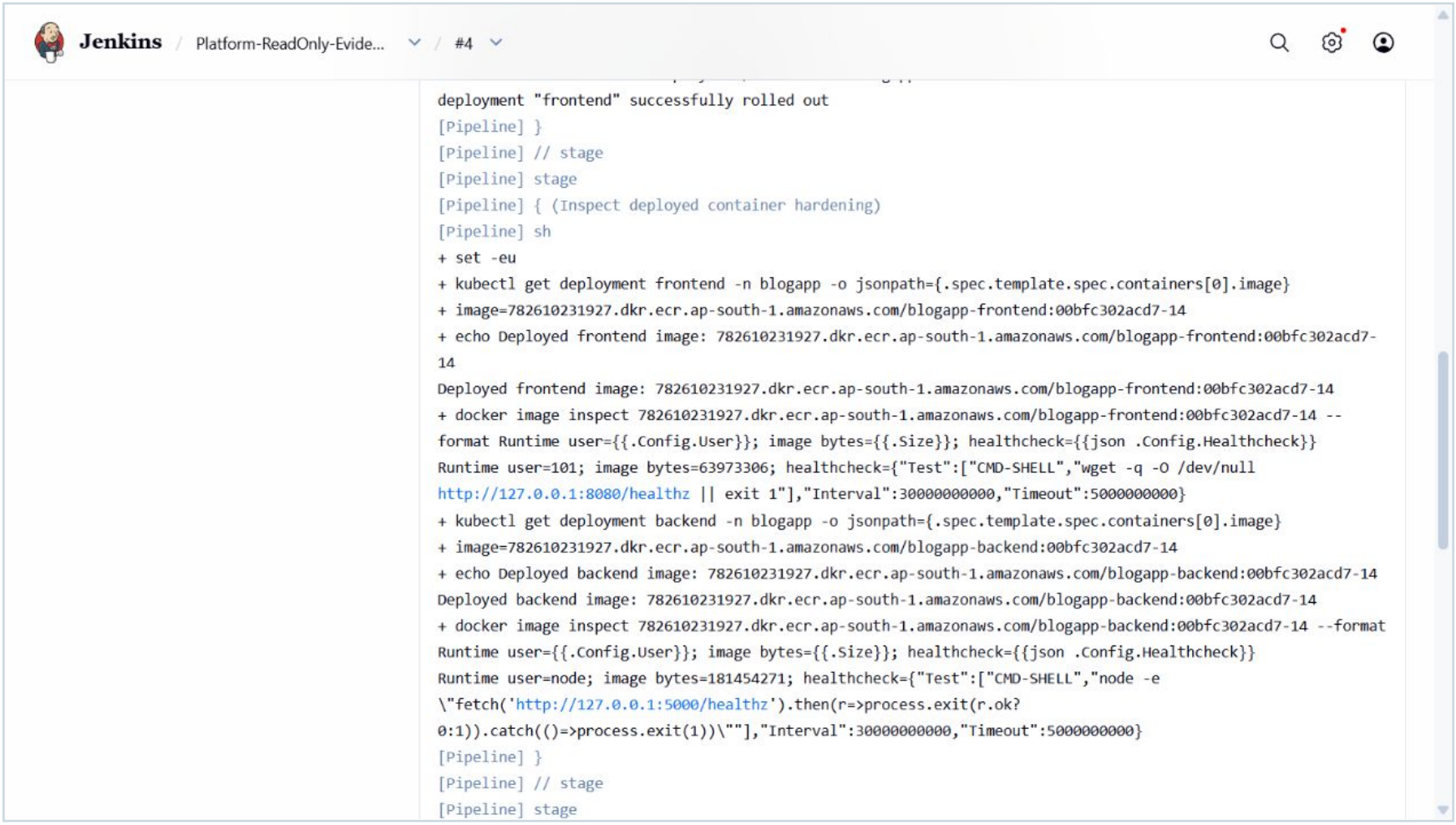
This page is rendered from data returned by the backend and stored in MySQL. The comment was created with an authenticated public API request. The same post and comment were verified after database pod recreation.



Post id 3 and its existing comment were retained. Original before/after record: raw/E09-database-persistence-test.json.

Built images run as non-root users

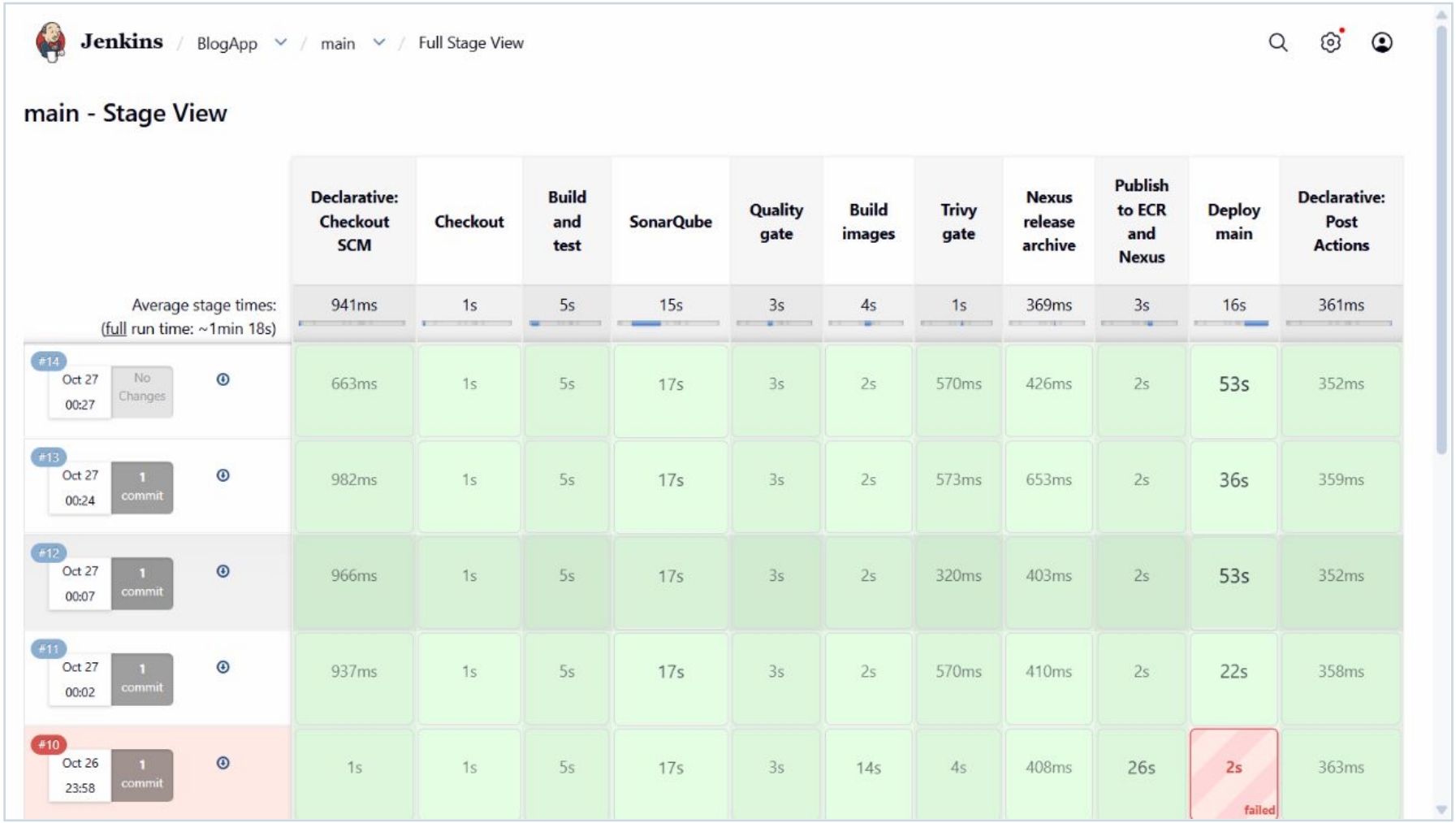
Frontend and backend use separate build/runtime stages, .dockerignore files and Docker health checks. Runtime packages were patched; unused Node package managers were removed from the backend image. Actual user IDs were frontend 101, backend 1000 and MySQL 999.



Real Jenkins image inspection for the deployed tag 00bfc302acd7-14. Uncompressed image sizes: frontend 63,973,306 bytes; backend 181,454,271 bytes.

All delivery stages succeeded

Acceptance build #14 completed successfully. Builds #11-#14 show successful delivery after setup corrections. Source commit changes are indexed automatically by the multibranch job; the #12 rolling acceptance run was triggered by a source commit.

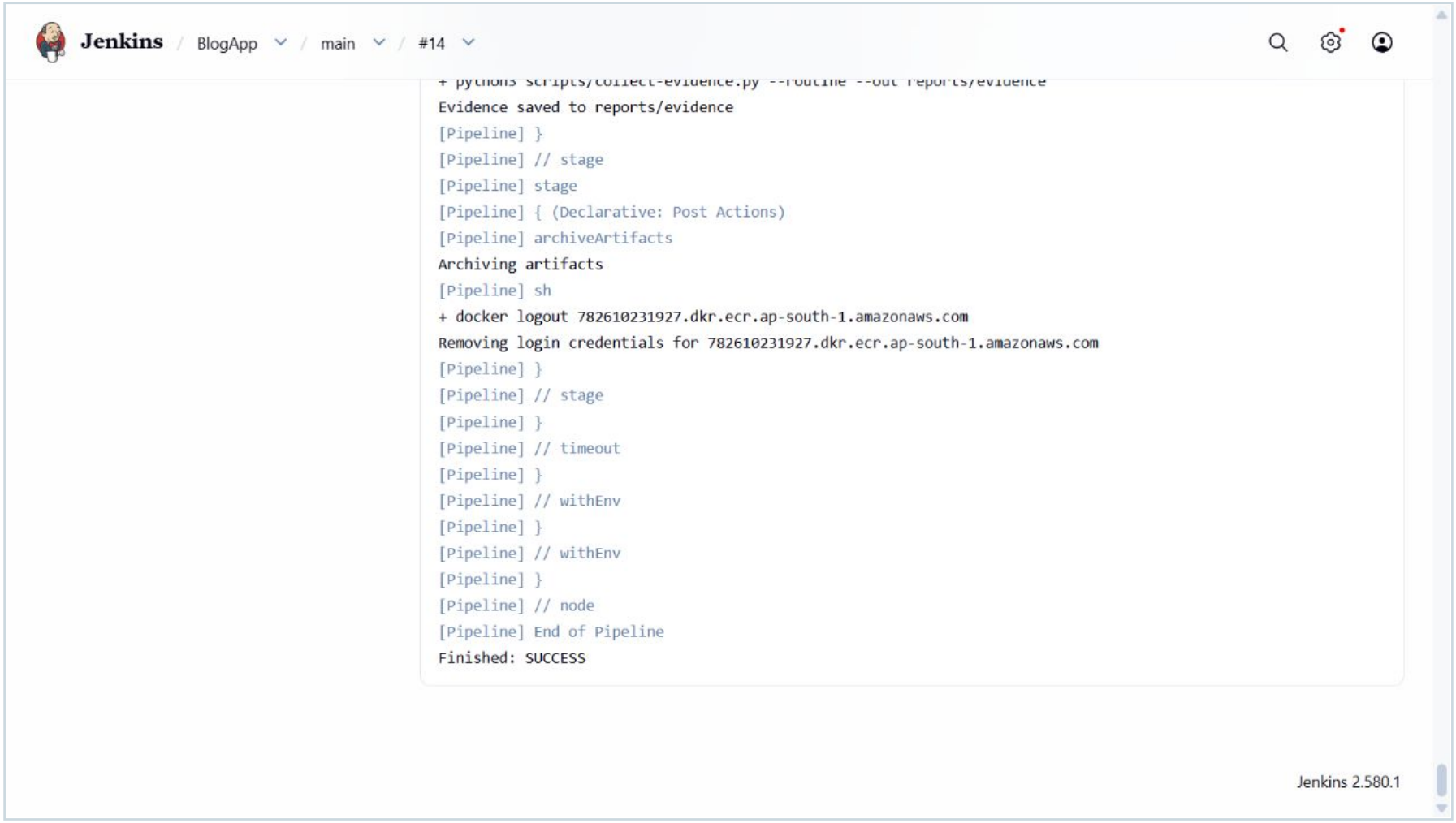


	Declarative: Checkout SCM	Checkout	Build and test	SonarQube	Quality gate	Build images	Trivy gate	Nexus release archive	Publish to ECR and Nexus	Deploy main	Declarative: Post Actions
Average stage times: (full run time: ~1min 18s)	941ms	1s	5s	15s	3s	4s	1s	369ms	3s	16s	361ms
#14 Oct 27 00:27 No Changes	663ms	1s	5s	17s	3s	2s	570ms	426ms	2s	53s	352ms
#13 Oct 27 00:24 1 commit	982ms	1s	5s	17s	3s	2s	573ms	653ms	2s	36s	359ms
#12 Oct 27 00:07 1 commit	966ms	1s	5s	17s	3s	2s	320ms	403ms	2s	53s	352ms
#11 Oct 27 00:02 1 commit	937ms	1s	5s	17s	3s	2s	570ms	410ms	2s	22s	358ms
#10 Oct 26 23:58 1 commit	1s	1s	5s	17s	3s	14s	4s	408ms	26s	2s failed	363ms

Actual Jenkins full stage view. Test, SonarQube, gate, Docker build, Trivy, Nexus archive, dual registry publishing and EKS deployment are green for #14.

Deployment and smoke validation completed

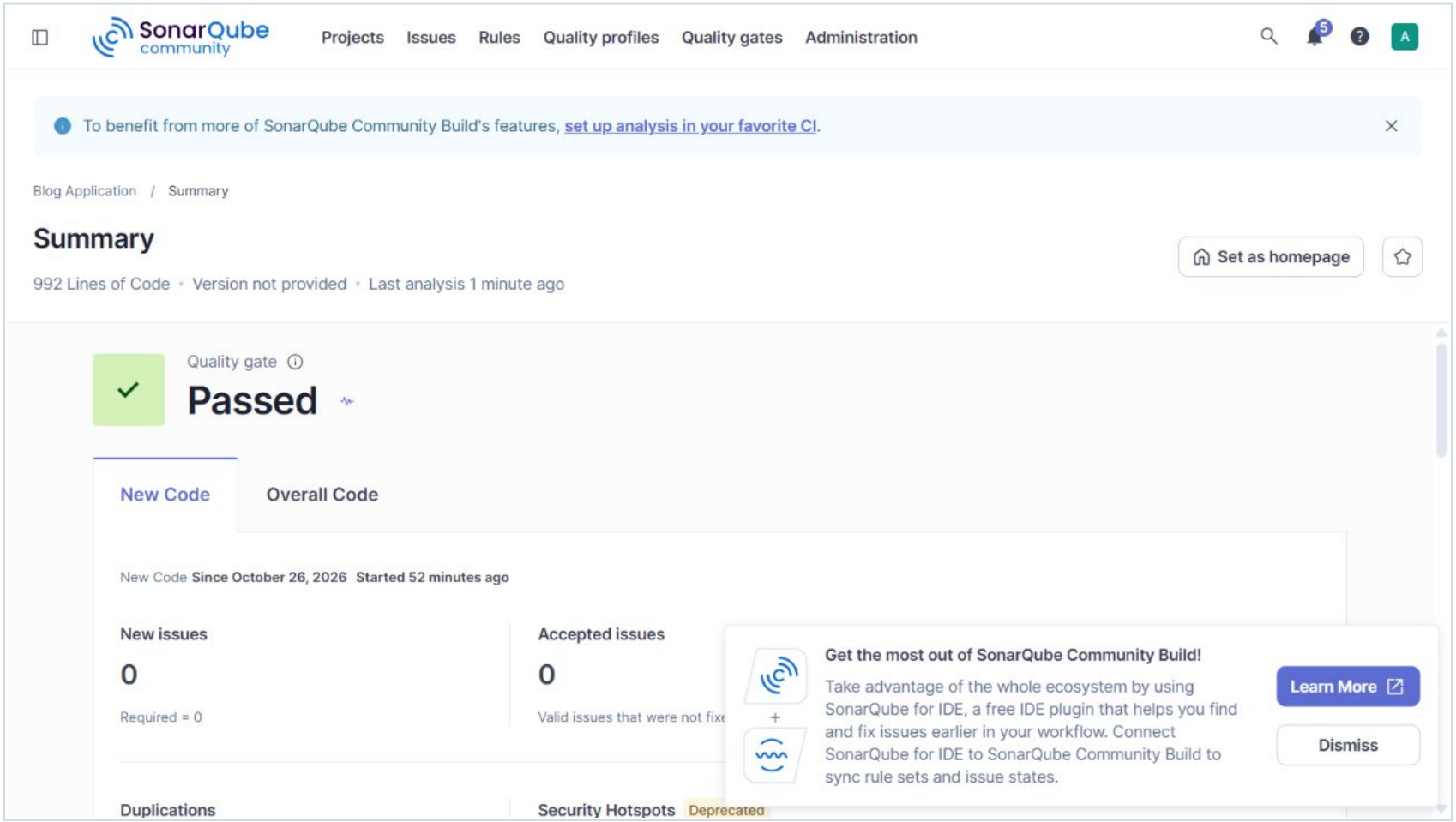
The console records rollout completion, public API write/read/comment checks, evidence collection and the final SUCCESS result. Raw console: raw/E03-jenkins-build-14-console.txt. Private operator build link: http://127.0.0.1:18080/job/BlogApp/job/main/14/



Actual #14 console final result. Unit tests: backend 7 + frontend 3; production frontend bundle built successfully.

Quality gate enforced for the specific analysis

The scanner communicates with Community Build 26.10.0.133078. The pipeline polls the submitted analysis ID and rejects a non-OK gate before image publication or deployment. Jenkins stays private without an incoming webhook.



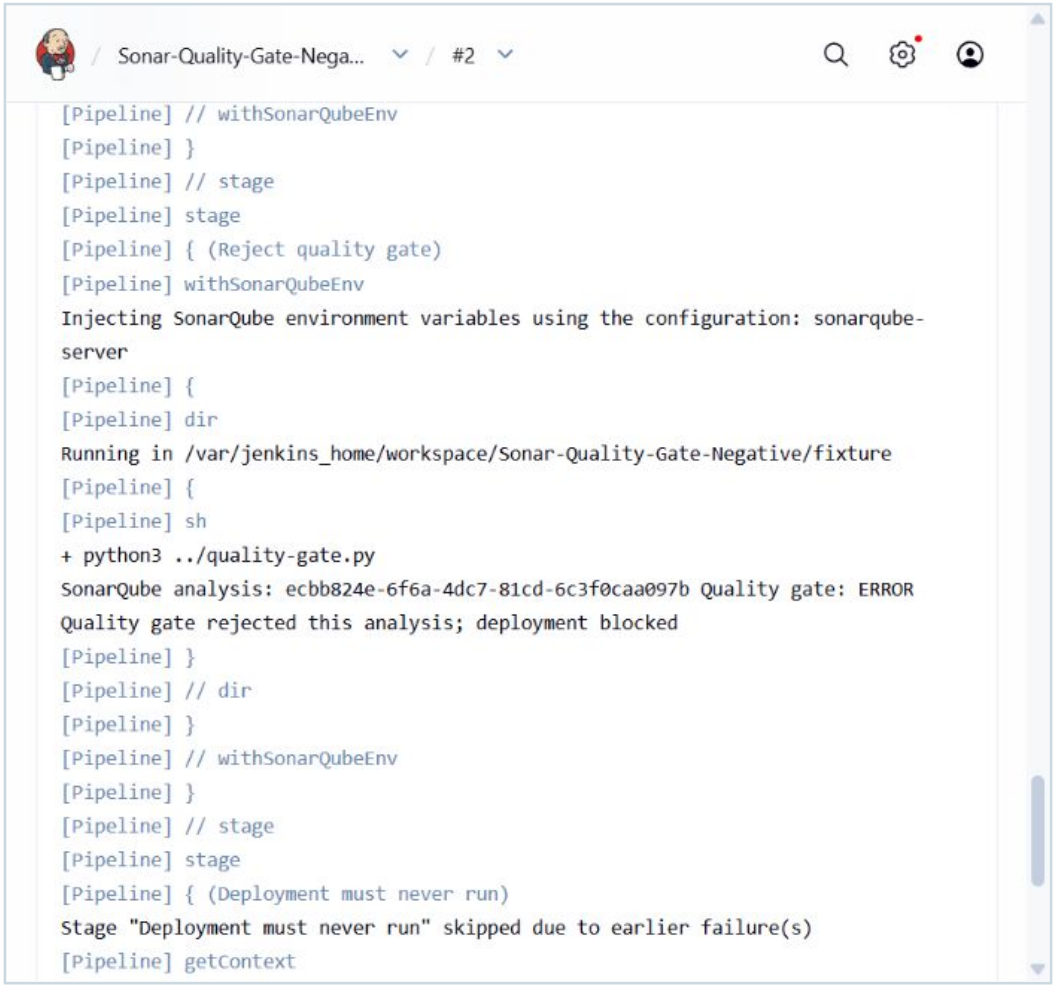
Main project new-code gate: Passed, zero new issues. Raw final analysis result: raw/E04-final-sonar-gate.json.

Limits: overall coverage is 40.6%; existing overall reliability rating C (21 findings), maintainability A (3 findings), security A (0 findings). No findings were waived. The gate is scoped to new code; Community Build has limited injection-vulnerability analysis.

E04 / NEGATIVE TEST

A failed quality gate blocks deployment

An isolated project intentionally omitted tests and used a separate coverage gate requiring 100%. It reported 0% coverage and ERROR. This did not modify or weaken the application project gate. The deployment stage was skipped.

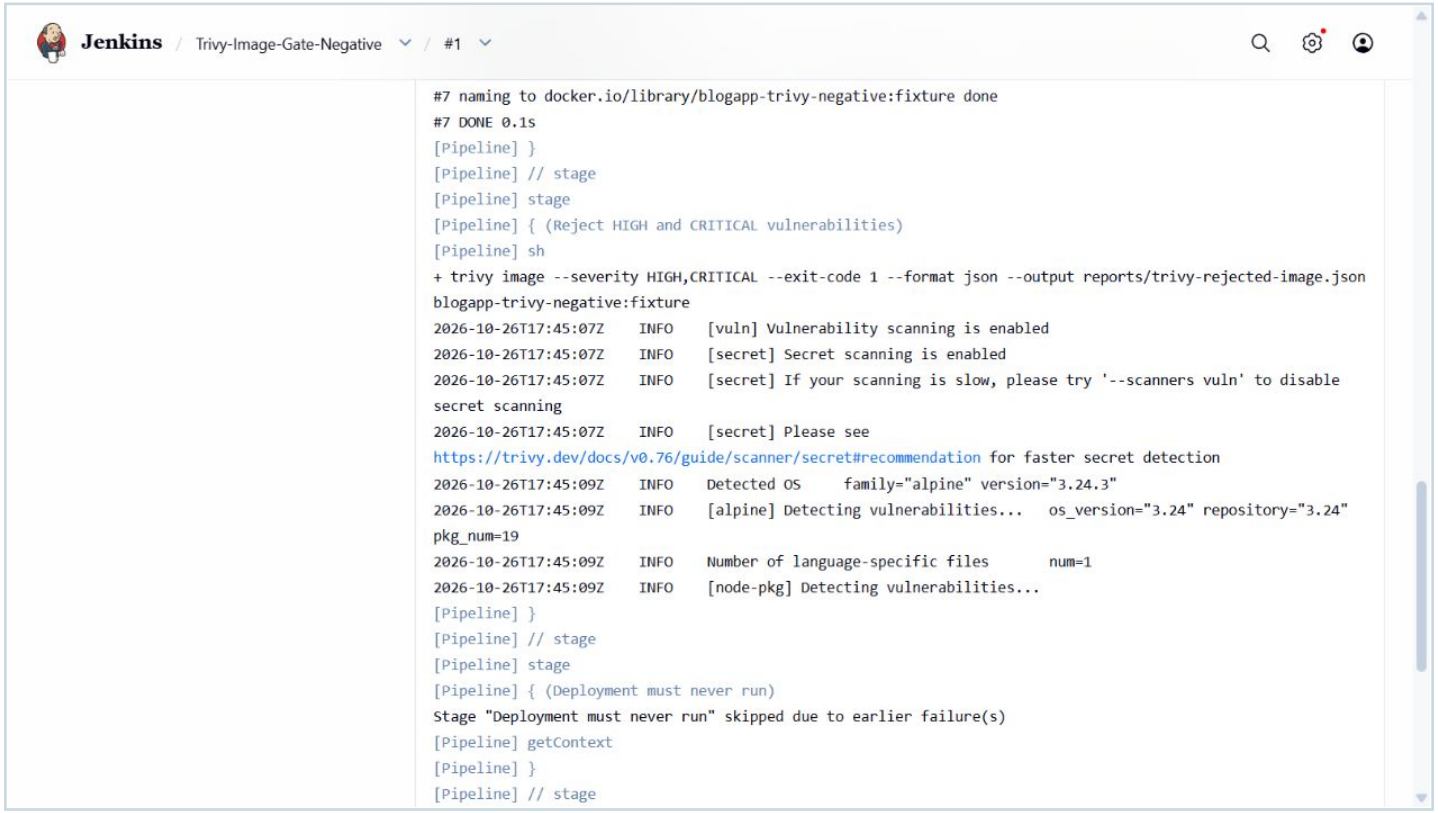


Sonar-Quality-Gate-Negative #2. Source definition: verification/cloud/SonarNegative.Jenkinsfile. Original result: raw/E04-sonar-negative-result.json.

Both final images passed the unchanged image gate

Image	HIGH vulnerabilities	CRITICAL vulnerabilities	HIGH/CRITICAL scan gate
frontend	0	0	PASS
backend	0	0	PASS

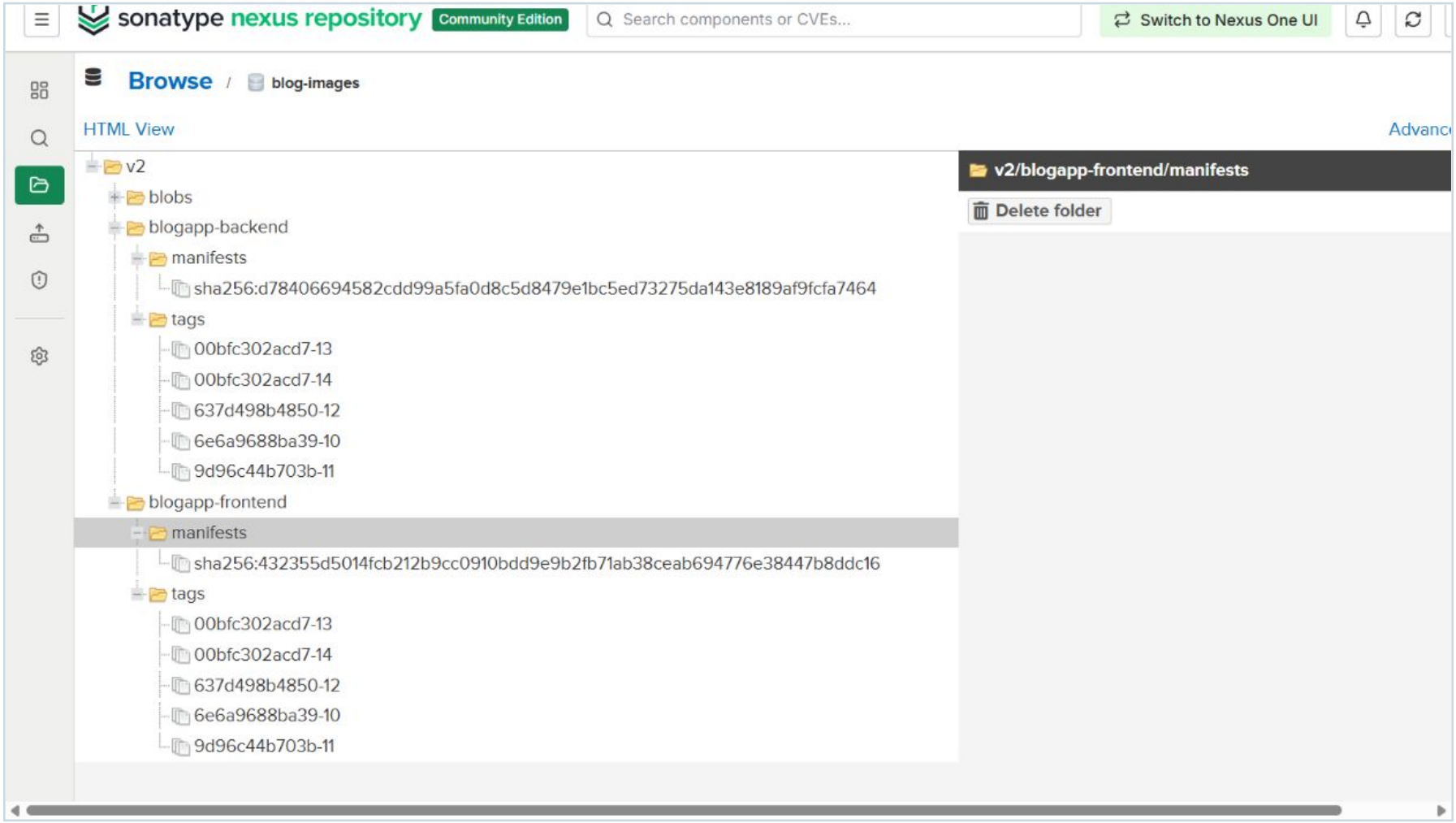
Trivy image scanning used --severity HIGH,CRITICAL --exit-code 1 before repository publication and EKS deployment. Findings from an earlier application release were remediated by patching Alpine runtime libraries and removing unused bundled package-manager dependencies. No --ignore-unfixed or severity downgrade was used.



Separate deliberately vulnerable Docker image: Trivy-Image-Gate-Negative #1. Image scan failed and deployment was skipped. Full vulnerability JSON is attached.

Versioned images and matching digests in Nexus

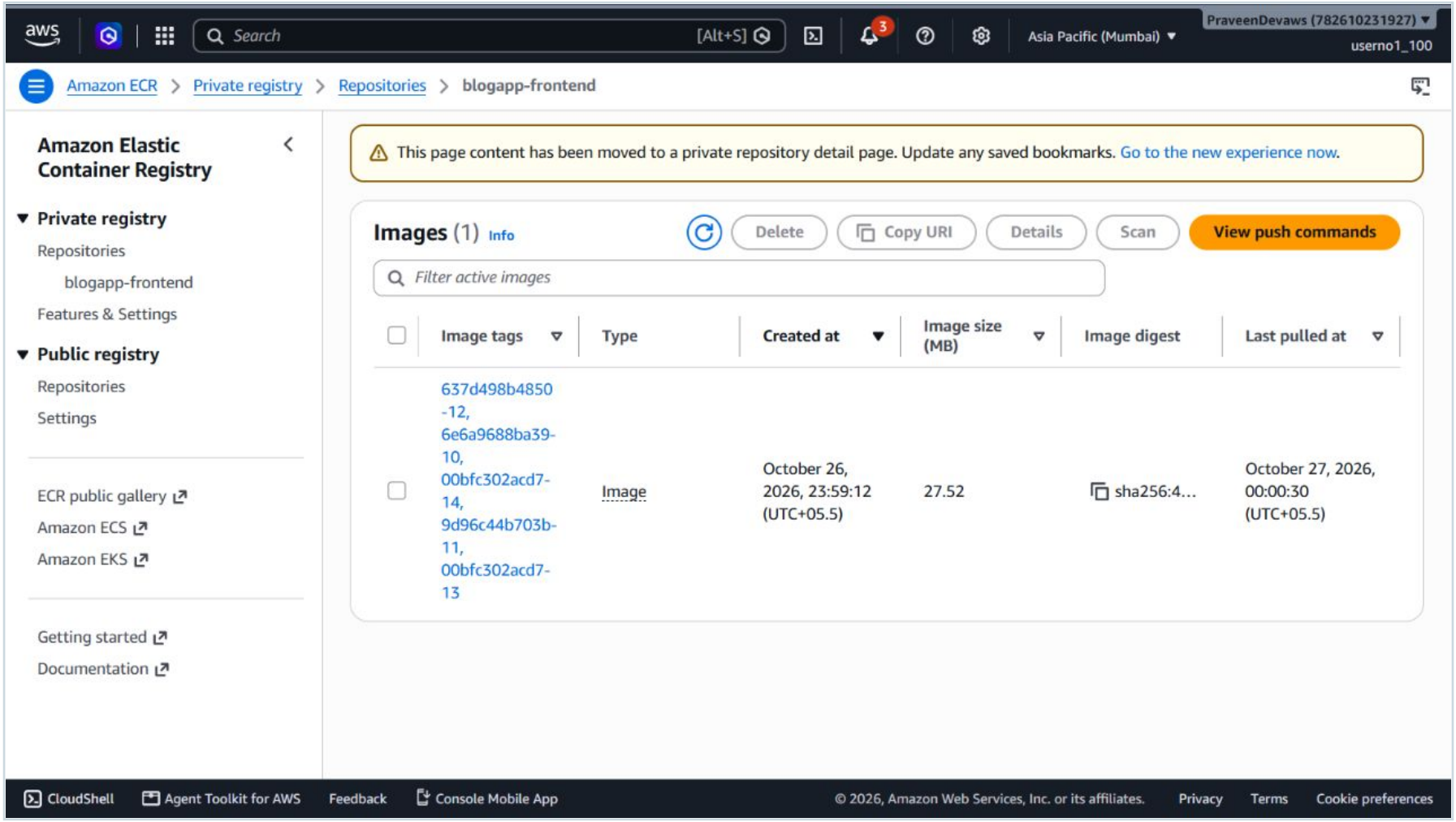
Nexus stores both frontend/backend image versions, with redeployment disabled, and a committed source archive. A repository-scoped deployment account publishes through a private TLS endpoint. Nexus now uses a protected project-specific encryption key; its health checks passed.



Nexus image tree shows 00bfc302acd7-14 for both components and the same content digests used by ECR. Raw Nexus component/archive metadata is attached.

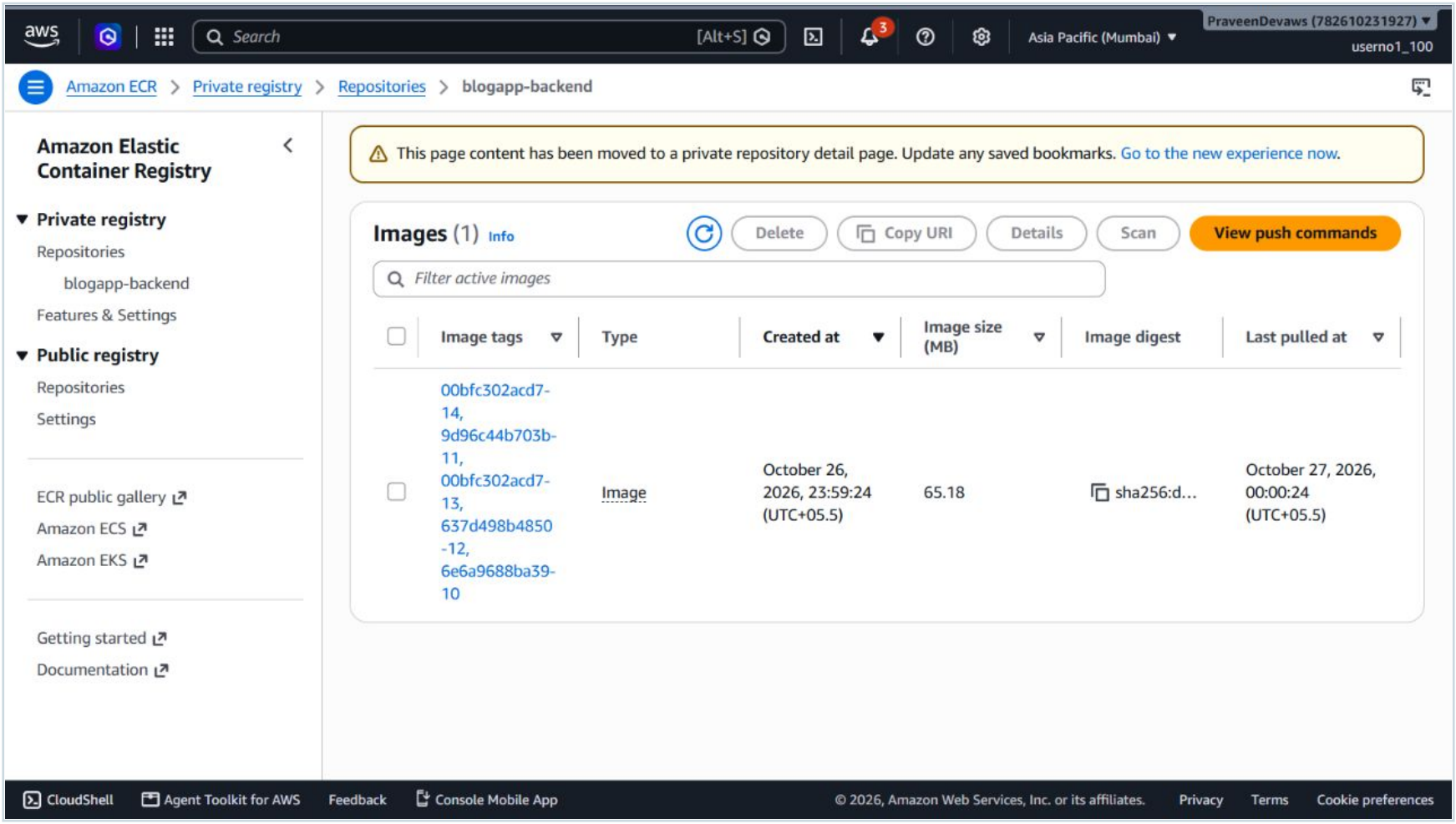
Versioned frontend image in the private ECR registry

ECR repositories use encryption, immutable tags and scan-on-push. EKS pulls from ECR. Different release tags may reference identical cached application content; the tags remain immutable and traceable to their build/source records.



ECR frontend repository includes acceptance tag 00bfc302acd7-14. Exact digest and registry record: raw/E05-ecr-final-images.json.

Versioned backend image in the private ECR registry

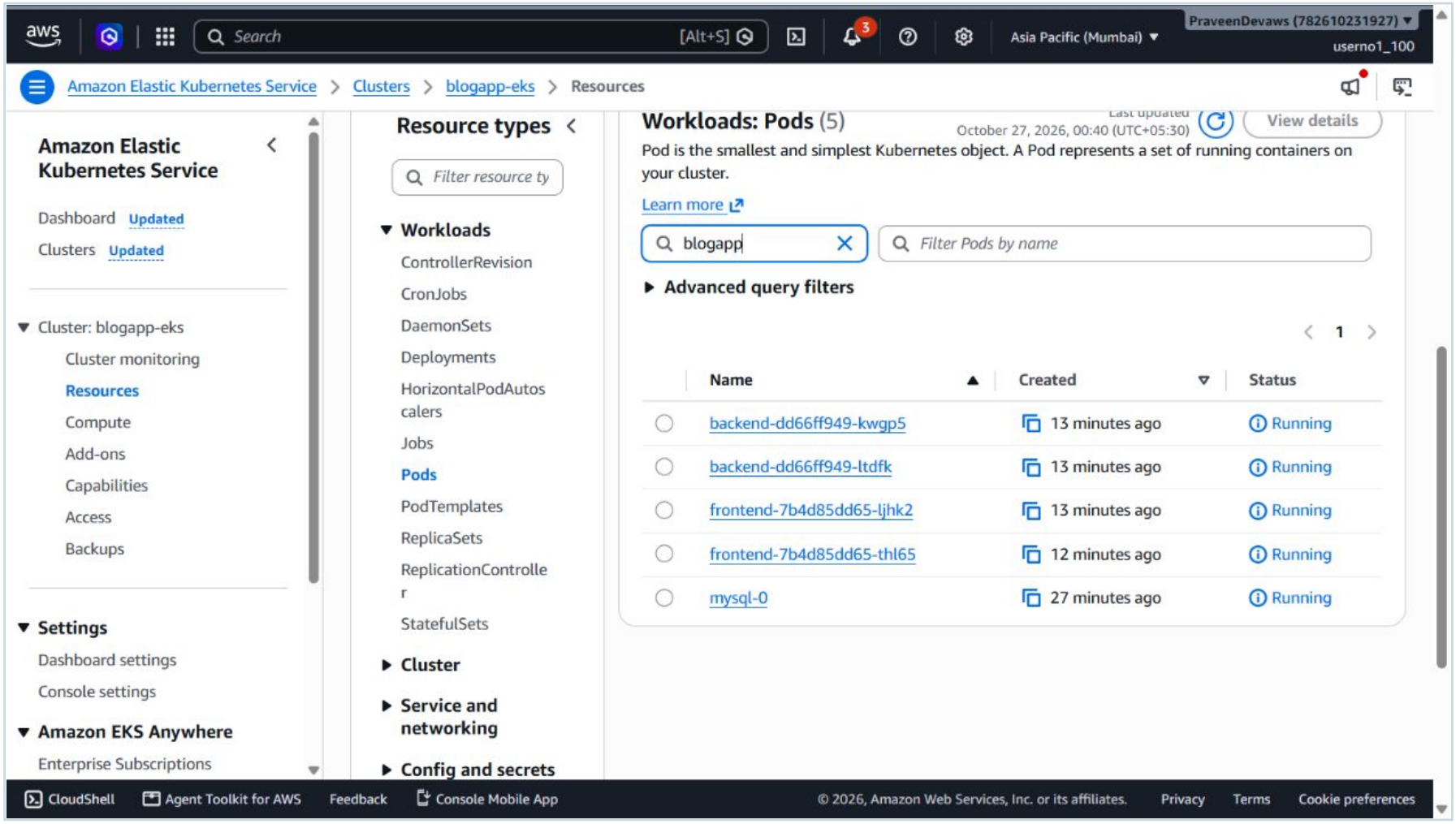


ECR backend repository includes acceptance tag 00bfc302acd7-14. The screenshot size is registry-compressed storage, not the uncompressed Docker runtime size.

Component	Digest shared by Nexus and ECR
frontend	sha256:432355d5014fcb212b9cc0910bdd9e9b2fb71ab38ceab694776e38447b8ddc16
backend	sha256:d78406694582cdd99a5fa0d8c5d8479e1bc5ed73275da143e8189af9fcfa7464

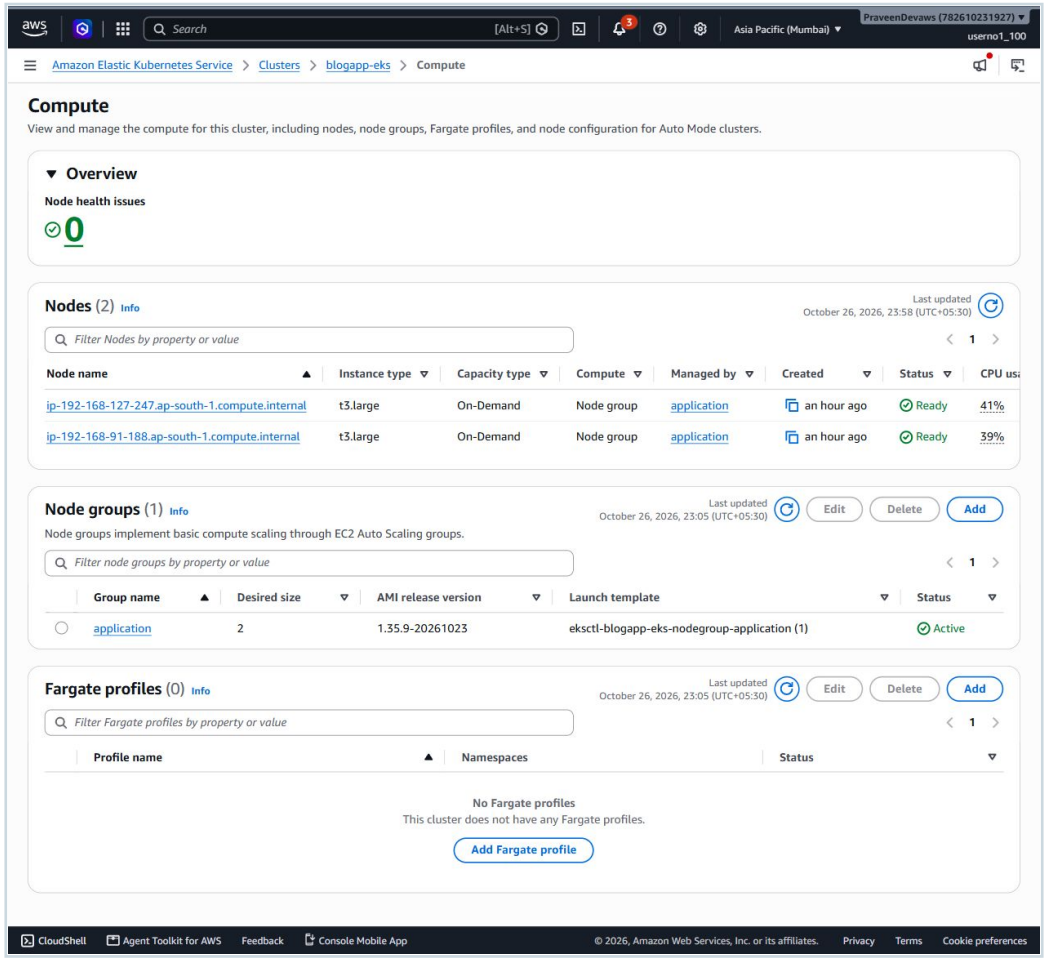
Two frontend pods, two backend pods and MySQL

Five application pods are Running. Frontend/backend Deployments each maintain two replicas. Readiness and liveness probes, requests/limits, rolling update maxUnavailable=0/maxSurge=1, PDBs and HPA definitions are included in k8s/. All three application Services remain internal.



Actual AWS EKS namespace pod view. Kubernetes configuration/runtime snapshots are attached under E07 and E03 in raw/.

Healthy worker nodes across two Availability Zones



Two t3.large workers in the managed application node group; zero node health issues. The full-resolution screenshot is included in the evidence folder.

The controller and managed add-ons were installed and verified, including EBS CSI permissions, metrics-server, ALB controller and VPC CNI policy enforcement. The cluster public API is restricted to a single operator CIDR; CI uses private connectivity.

Pod recreation retained data; isolated restore matched

Persistence check	Before	After
Database pod UID	877364e1-3435-4244-a544-2248d9224edf	1fc06f12-c7d4-4693-86e5-1e2b4e647764
Persistent volume	pvc-f58922e6-d455-4c9d-8306-33fa70441279	pvc-f58922e6-d455-4c9d-8306-33fa70441279
Stored post id	3	3
Stored comments	1	1
Recovery time	Replacement pod + public read-back	28.98 seconds

The pod UID changed while the volume and application content stayed the same. This was an intentional single-database maintenance test; the 28.98-second recovery is not a claim of zero-downtime database high availability.

Backup / restore	Source blogdb	Isolated blogdb_restore_proof
Users	3	3
Posts	5	5
Comments	5	5

The 7,849-byte logical backup was restored into a separate database. Its SHA-256 is 50b4581a775b4b6c5db6c0a2a5ed01e87c3ee23793ca33c74e45c2ce5b6ad11a. The SQL backup contains private account data and is retained privately. Later smoke runs add records; the table counts above are the captured restore snapshot.

196 successful requests during a commit-triggered rollout

Measured result	Value
Release under continuous sampling	BlogApp/main #12 - SUCCESS; source 637d498b4850cb8baac277fd4286180ca7f14f86
Public requests sampled	196
HTTP 200 responses	196
Failures	0
Maximum observed request latency	1593.0 ms
Validation data	raw/E10-rolling-traffic-test.json and E10-rolling-traffic-samples.csv

The initial rollout had exposed a transient 502. The implementation then added ALB pod readiness gates, 20-second target deregistration, 25-second frontend preStop, 10-second backend preStop and a 60-second termination grace period. The next commit-triggered rollout passed the measured traffic test.

The 196-request sample supports observed availability during that test. It is not a universal downtime or latency guarantee. Acceptance release #14 subsequently used the same configured rollout controls and passed public smoke validation.

Configuration values are separated from images

Input	Production mapping	Evidence
frontend/.env.example: VITE_API_URL=/api	Relative browser API path; Nginx routes to backend-service:5000	Frontend bundle + Nginx config
PORT / DB_HOST / DB_PORT / DB_USER / DB_NAME	backend-config ConfigMap	raw/E11-config-map.json
CORS_ORIGIN	Actual public HTTPS application origin	raw/E11-config-map.json
DB_PASSWORD / JWT_SECRET	blog-secrets Secret; generated once; Secrets Manager copy	Only metadata and references exported
MYSQL_ROOT_PASSWORD	Database Secret reference; not in schema or source	k8s/mysql.json
Pipeline credentials and Nexus encryption key	Protected CI host/Jenkins credential storage; TLS private CA trust	No key/password material in the public package

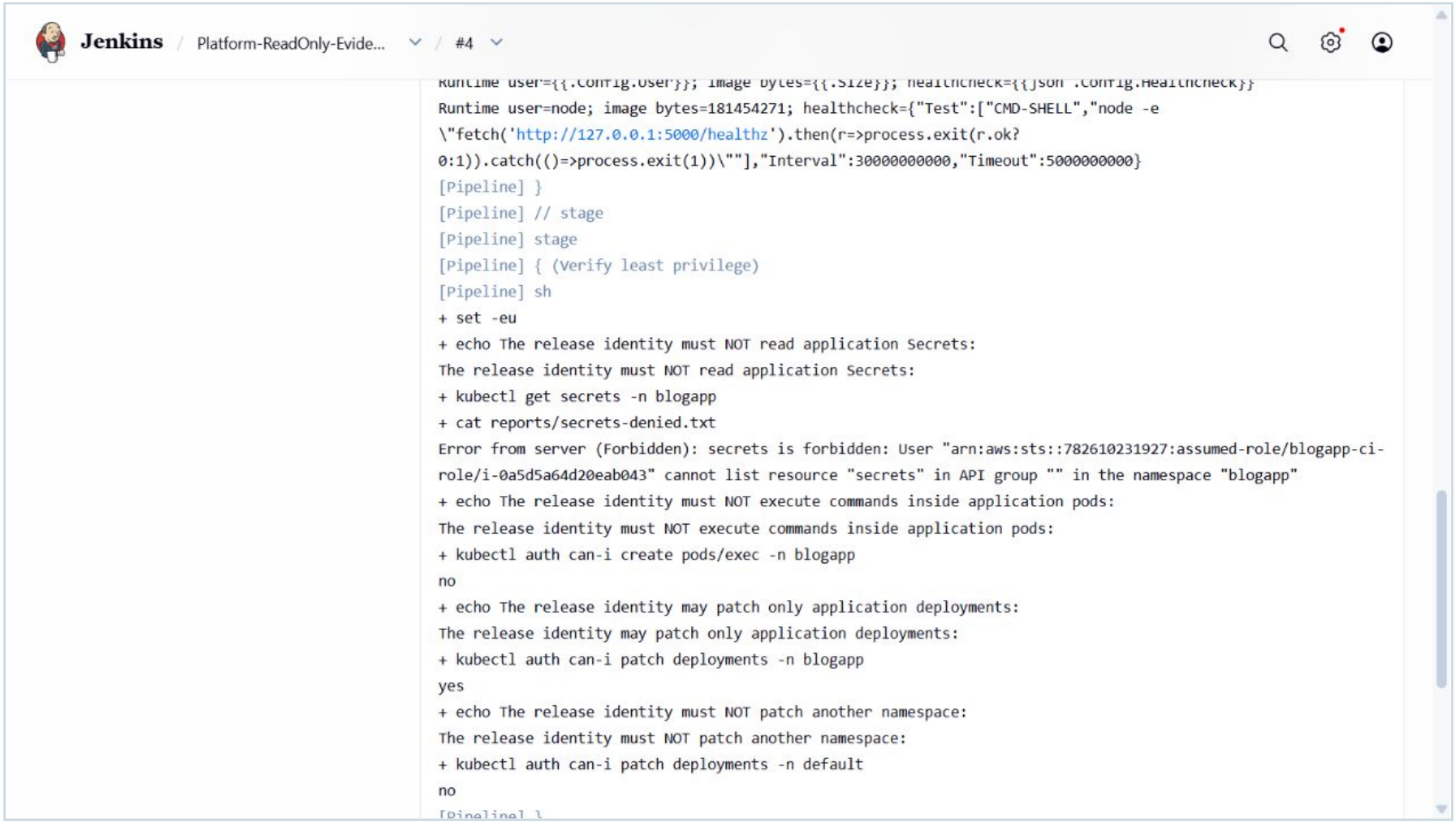
The .env.example secret fields are empty. The real local environment, AWS session files, CI passwords, TLS private keys and SQL backup are excluded from delivery. Secret values were injected through protected API/stdin operations and were not embedded in Dockerfiles or Kubernetes source manifests.

```
NAME           TYPE     CREATED
blog-secrets   Opaque   2026-10-26T17:21:37Z
```

Source and exported evidence were reviewed for credential patterns before publication. This is a scoped credential review, not a claim that automated pattern matching alone proves the absence of every possible secret.

The release role cannot read Secrets or execute in pods

The CI IAM role has ECR authorization plus publishing permissions only for the two application repositories, EKS DescribeCluster for blogapp-eks, and the SSM managed-instance role. Its Kubernetes group is bound to the namespace-scoped blog-deployer Role.



Actual Forbidden response for Secret listing, pod execution denied, application Deployment patch allowed, default-namespace patch denied. The restricted release role was retained.

Intended traffic allowed; unrelated client rejected

Actual test	Expected	Observed
Unrelated pod cannot reach MySQL	Denied	PASS; exit 1; 9.02 s
Unrelated pod cannot reach backend	Denied	PASS; exit 1; 9.84 s
Frontend can reach backend	Allowed	PASS; exit 0; 5.91 s
Backend can reach MySQL	Allowed	PASS; exit 0; 7.5 s

A temporary unrelated busybox client was created in blogapp. TCP access to mysql-service:3306 timed out and backend readiness access was blocked. Existing frontend and backend containers then verified their intended connections. The test client was removed.

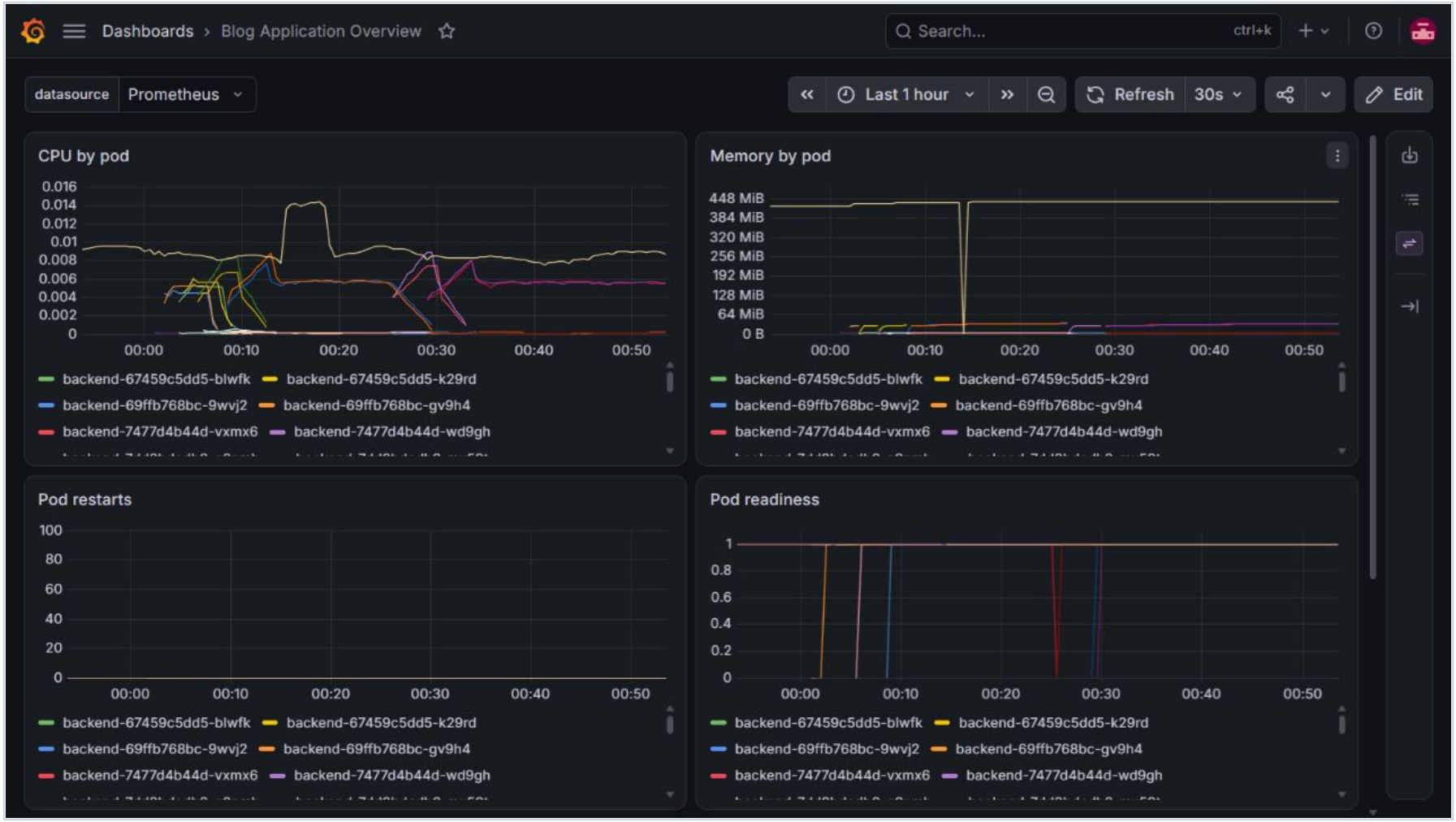
mysql-ingress selects MySQL and allows TCP 3306 only from app=backend pods in the application namespace. backend-ingress allows application traffic from frontend pods and metrics traffic from the monitoring namespace. VPC CNI policy enforcement was enabled and the negative tests confirmed it was effective.

```
Evidence: raw/E12-network-policy-tests.json
Configuration: raw/E12-network-policies.json
CNI and security-group metadata: raw/E11-security-platform.json
```

Private CI has no inbound security-group rules. The internal ALB accepts port 80 from the API Gateway VPC Link security group, while the application database is not externally exposed.

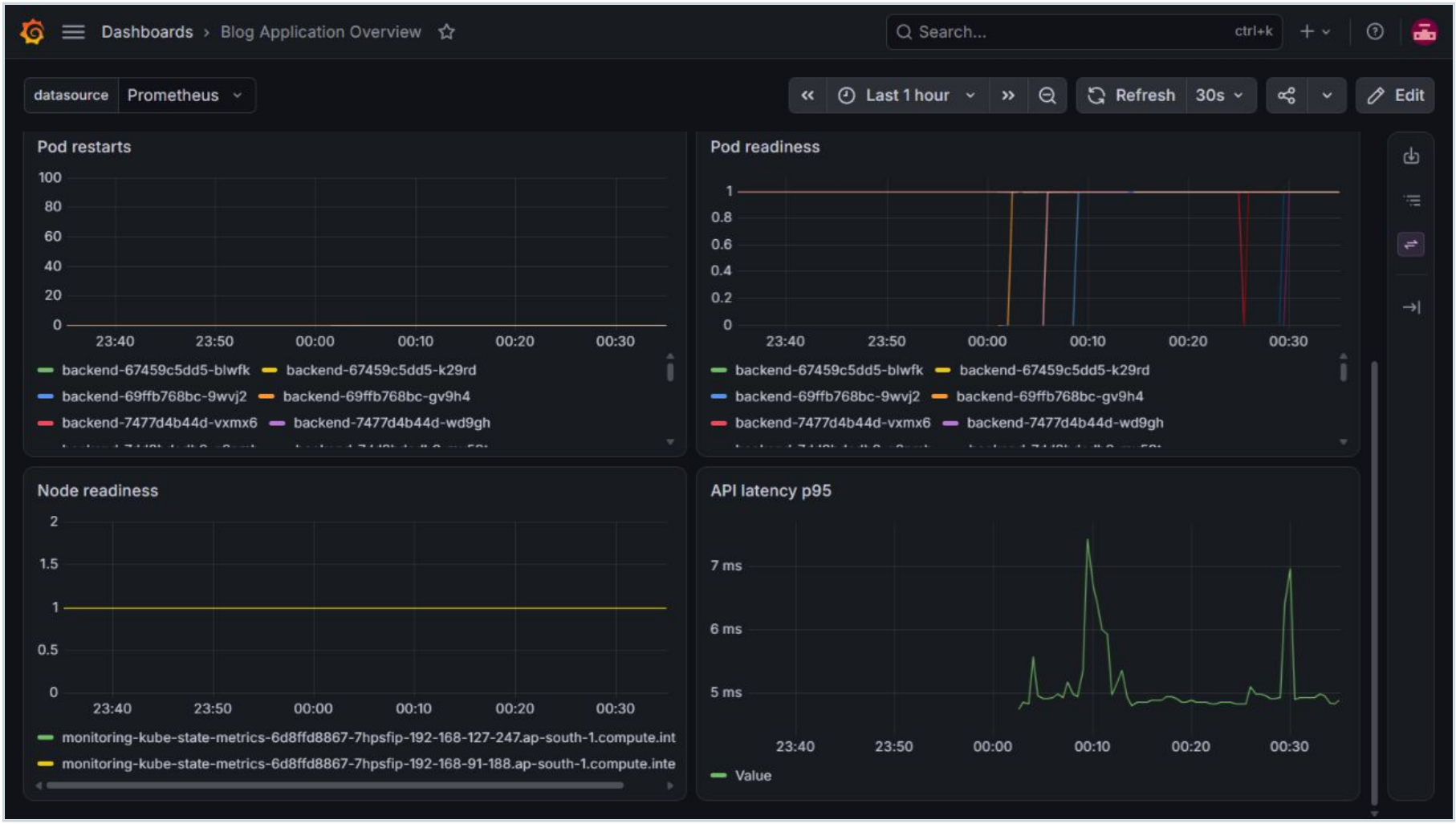
CPU, memory, restarts and readiness graphs

Prometheus, Grafana and Alertmanager run with persistent volumes. The application ServiceMonitor scrapes backend metrics. All six dashboard expressions were queried successfully and returned data. This capture includes historical pods from actual rolling releases.



Actual Grafana application dashboard. Original query verification: raw/E13-monitoring-verification.json.

Node readiness and API latency are populated

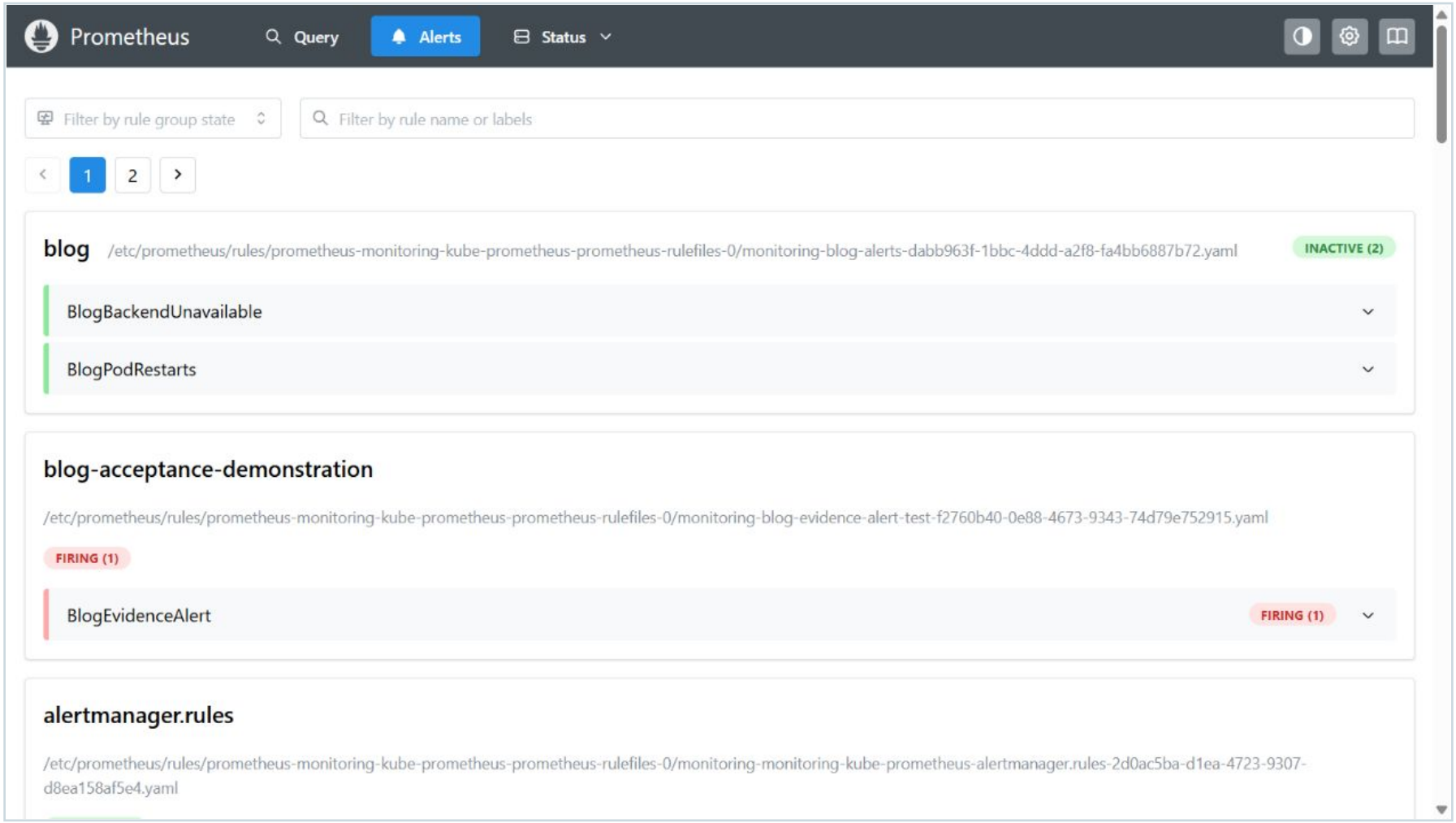


The two Ready-node series overlap at 1. API latency p95 is based on the backend request histogram. These metrics do not represent internet round-trip latency.

Dashboard definitions are in monitoring/dashboard.json. Infrastructure/cluster metrics come from node exporter, kube-state-metrics and kubelet scraping; backend application metrics come from its /metrics endpoint.

A synthetic test alert fired, then was removed

BlogEvidenceAlert deliberately evaluated vector(1) for 15 seconds, with purpose=acceptance-test. The screenshot shows the controlled alert firing while the real blog availability/restart rules are inactive. The test rule was then deleted and zero active synthetic alerts were verified.



Actual Prometheus alert page. Original firing and cleanup records: raw/E13-alert-firing-test.json and E13-alert-resolved-test.json. No external email notification is claimed.

Frontend, backend and database log records

Application logging is demonstrated using actual kubectl logs exports collected with the existing operator identity. These excerpts are recorded text from the attached raw files, not screenshots of a simulated terminal. Control-plane API, audit and authenticator logs are additionally enabled in CloudWatch with seven-day retention.

```
KUBERNETES APPLICATION LOGGING - RECORDED EXECUTION
Captured UTC: 2026-10-26T19:18:54.062950+00:00
Existing project operator access; original command output follows.

COMMAND: kubectl logs deployment/frontend -n blogapp --timestamps --tail=3
2026-10-26T19:17:44.718589734Z 192.168.126.189 - - [26/Oct/2026:19:17:44 +0000] "GET /assets/index-nQt1e6Rd.js HTTP/1.1" 304 0 "https://1c3j649rgg.execute-api.ap-south-
1.amazonaws.com/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/156.0.0.0 Safari/537.36" "192.168.90.14"
2026-10-26T19:17:44.721793816Z 192.168.85.174 - - [26/Oct/2026:19:17:44 +0000] "GET /assets/index-osMMORaN.css HTTP/1.1" 304 0 "https://1c3j649rgg.execute-api.ap-south-
1.amazonaws.com/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/156.0.0.0 Safari/537.36" "192.168.90.14"
2026-10-26T19:17:44.847656163Z 192.168.85.174 - - [26/Oct/2026:19:17:44 +0000] "GET /favicon.ico HTTP/1.1" 304 0 "https://1c3j649rgg.execute-api.ap-south-1.amazonaws.com/"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/156.0.0.0 Safari/537.36" "192.168.108.199"
EXIT CODE: 0

COMMAND: kubectl logs deployment/backend -n blogapp --timestamps --tail=3
2026-10-26T19:19:01.002532454Z ::ffff:192.168.127.247 - - [26/Oct/2026:19:19:01 +0000] "GET /healthz HTTP/1.1" 200 15 "-" "kube-probe/1.35"
2026-10-26T19:19:01.019504651Z ::ffff:192.168.127.247 - - [26/Oct/2026:19:19:01 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.35"
2026-10-26T19:19:06.019273316Z ::ffff:192.168.127.247 - - [26/Oct/2026:19:19:06 +0000] "GET /readyz HTTP/1.1" 200 18 "-" "kube-probe/1.35"
EXIT CODE: 0

COMMAND: kubectl logs mysql-0 -n blogapp --timestamps --tail=3
2026-10-26T18:43:58.371113007Z 2026-10-26T18:43:58.370743Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is
accessible to all OS users. Consider choosing a different directory.
2026-10-26T18:43:58.439041345Z 2026-10-26T18:43:58.438761Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port: 33060, socket:
/var/run/mysqld/mysqlx.sock
2026-10-26T18:43:58.439070956Z 2026-10-26T18:43:58.438953Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.4.13'  socket:
'/var/run/mysqld/mysqld.sock'  port: 3306  MySQL Community Server - GPL.
EXIT CODE: 0
```

Genuine browser view of the recorded kubectl command output. Source: raw/E14-logging-execution.txt. This is a saved execution record, not a simulated terminal UI.

An actual CreateContainerConfigError identified a symbolic non-root user; the backend was fixed with numeric UID 1000. Rollout validation exposed a transient 502; readiness/draining was corrected and the measured rerun had zero errors. Both are examples of diagnosing and correcting deployment issues from execution evidence.

What to open for evaluation

ID	Proof	Files in cloud-execution-evidence/
E01	Source revision	E01-github-release-source.png; pinned GitHub source commit
E02	Container builds / hardening	E02-deployed-image-hardening.png; E02-live-container-uids.json
E03	Complete pipeline	E03-jenkins-successful-stage-view.png; build-14 console/stages JSON
E04	SonarQube + rejected gate	Sonar passed/rejected screenshots; final and negative gate JSON
E05	Nexus + ECR versions	Nexus and both ECR screenshots; final image, digest and archive metadata
E06	Trivy image gates	Rejected image screenshot; both final image JSON reports; negative fixture report
E07	Live EKS workloads	Five-pod and worker-node screenshots; deployment/runtime snapshots
E08	Three-tier application	Home + post/comment screenshots; application UAT JSON
E09	Database persistence	Retained-post screenshot; before/after pod UID and volume record
E10	Rolling traffic	196-request JSON summary and per-request CSV
E11	IAM / RBAC / config	Forbidden-access screenshot; Secret metadata, Role and IAM records
E12	Network restrictions	Positive/negative execution results; effective NetworkPolicy JSON
E13	Monitoring / alert	Two Grafana screenshots; Prometheus firing screenshot; query/cleanup JSON
E14	Logging	Raw frontend/backend/MySQL logs; initialization log; CloudWatch settings
E15	Backup and restore	Isolated restore result with matching table counts and backup hash

Submission links and honest limits

Application: https://1c3j649rgg.execute-api.ap-south-1.amazonaws.com
Repository: https://github.com/praveenpreni/BlogReact
Document: Cloud_Execution_Report.pdf on GitHub
Acceptance source: 00bfc302acd73933912fd12579b72885081d22cb

Scope / limit	Meaning for this submission
MySQL and CI availability	Single database instance and single CI host. Persistence/restore is proven; database or CI high availability is not.
Quality coverage	New-code gate passed. Overall coverage remains 40.6%, with 21 existing reliability findings; Community Build has limited injection analysis.
Private tool links	Jenkins/Sonar/Nexus/Grafana localhost views need authenticated operator tunnels. The public review link is the application HTTPS URL.
Acceptance snapshot	Release #14 and immutable tag 00bfc302acd7-14 were recorded. Documentation publication can advance main; accepted execution source and image records remain pinned.
Backups and credentials	The SQL backup and all actual passwords, tokens, session files and private keys remain private. Only safe result metadata is attached.
Operating costs and cleanup	Resources remain running for tutor review as requested; AWS charges continue. Remove project resources after approval, including retained EBS volumes, NAT, CI, EKS, API Gateway, registries, IAM and logs.

The earlier 22 October native local report is historical and is not used to claim Docker, Nexus, SonarQube or EKS execution. This cloud report includes the subsequent real deployment and its evidence.